1.

a. $H(z) = 1 - pz^{-3}$ $x(n)$ is the input for $d(n)$

ELEN 133
HW 8

Dylan Thornburg

$$\frac{d(z)}{x(z)} = 1 - pz^{-3}$$

$$d(z) = X(z) - pz^{-3}X(z)$$

$$\boxed{d(n) = x(n) - p\,x(n-3)}$$

b. needs to match $1 - pz^{-3}$

$$\boxed{w = [1, 0, 0, -p, 0, 0, 0, 0]^T \qquad e(n) = 0}$$

← we get everything.

c. Now we miss the $-p$

$$\boxed{w = [1, 0, 0]^T \qquad e(n) = -p\,x(n-3)}$$

2.

a. $H(z) = \dfrac{1}{1 - pz^{-3}}$

$$d(z) - pz^{-3}d(z) = x(z)$$

$$d(n) - p\,d(n-3) = x(n)$$

$$\boxed{d(n) = x(n) + p\,d(n-3)}$$

b. $H(z) = \dfrac{1}{1 - pz^{-3}}$ expands to

$$h(n) = \delta(n) + p\delta(n-3) + p^2\delta(n-6) + \dots$$

$$\boxed{\begin{aligned} &w = [1, 0, 0, p, 0, 0, p^2, 0] \\ &e(n) = p^3 d(n-9) + p^4 d(n-12) + \dots \end{aligned}}$$

c.

$$\boxed{\begin{aligned} &w = [1, 0, 0] \\ &e(n) = p\,d(n-3) + p^2 d(n-6) + \dots \end{aligned}}$$

3.

$x(0) = 0,\ x(1) = 1,\ x(2) = -1$ → repeats

$d(0) = 2,\ d(1) = -1,\ d(2) = -1$ → repeats

a. $x_n = \begin{bmatrix} x(n) \\ x(n-1) \\ x(n-2) \end{bmatrix}$ $\quad x_0 = \begin{bmatrix} x(0) \\ x((0-1)\%3) \\ x((0-2)\%3) \end{bmatrix} = \begin{bmatrix} x(0) \\ x(2) \\ x(1) \end{bmatrix}$

$$\boxed{\begin{aligned} &x_0 = \begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix} \qquad x_2 = \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix} \\ &x_1 = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} \end{aligned}}$$

b. $\begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}[0\ -1\ 1] = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & -1 \\ 0 & -1 & 1 \end{bmatrix} = x_0 x_0^T = R_0$

$\begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}[1\ 0\ -1] = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix} = x_1 x_1^T = R_1$

$\begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix}[-1\ 1\ 0] = \begin{bmatrix} 1 & -1 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} = x_2 x_2^T = R_2$

c. $R = (R_0 + R_1 + R_2)$

$$\boxed{R = \begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix}}$$

d.

$d(0)x_0 = 2\begin{bmatrix}0\\-1\\1\end{bmatrix} = \begin{bmatrix}0\\-2\\2\end{bmatrix}$ $\quad d(1)x_1 = -1\begin{bmatrix}1\\0\\-1\end{bmatrix} = \begin{bmatrix}-1\\0\\1\end{bmatrix}$ $\quad d(2)x_2 = -1\begin{bmatrix}-1\\1\\0\end{bmatrix} = \begin{bmatrix}1\\-1\\0\end{bmatrix}$

e. $\left(\begin{bmatrix}0\\-2\\2\end{bmatrix} + \begin{bmatrix}-1\\0\\1\end{bmatrix} + \begin{bmatrix}1\\-1\\0\end{bmatrix}\right) = \boxed{\begin{bmatrix}0\\-3\\3\end{bmatrix} = P}$

$-(2+2+2) = -6$

$\begin{bmatrix}2&-1&-1\\-1&2&-1\\-1&-1&2\end{bmatrix} \to \begin{bmatrix}2&-1&-1\\0&\frac{3}{2}&-\frac{3}{2}\\0&-\frac{3}{2}&\frac{3}{2}\end{bmatrix}$

f. $w = R^{-1}P$ $\quad R = \begin{bmatrix}2&-1&-1\\-1&2&-1\\-1&-1&2\end{bmatrix}$

$(8-1-1) = 6$

$6-6=0$ no det and $\to$

$\begin{bmatrix}2&-1&-1\\0&\frac{3}{2}&-\frac{3}{2}\\0&0&0\end{bmatrix} \to \begin{bmatrix}2&0&-2\\0&\frac{3}{2}&-\frac{3}{2}\\0&0&0\end{bmatrix}$

$\begin{bmatrix}1&0&-1\\0&1&-1\\0&0&0\end{bmatrix}$ R not the identity matrix. Not invertible

we can't compute weights b/c R is not invertible

4.

a. $x_0 = \begin{bmatrix}0\\-1\end{bmatrix}, x_1 = \begin{bmatrix}1\\0\end{bmatrix}, x_2 = \begin{bmatrix}-1\\1\end{bmatrix}$

$R_0 = \begin{bmatrix}0&0\\0&1\end{bmatrix}$ $\quad R_2 = \begin{bmatrix}1&-1\\-1&1\end{bmatrix}$; $\quad R = \begin{bmatrix}2&-1\\-1&2\end{bmatrix}$

$R_1 = \begin{bmatrix}1&0\\0&0\end{bmatrix}$ $\quad P = \begin{bmatrix}0\\-3\end{bmatrix}$

$R^{-1} = \begin{bmatrix}2&1\\1&2\end{bmatrix} \cdot \frac{1}{(4-1)} = \begin{bmatrix}2&1\\1&2\end{bmatrix} \cdot \frac{1}{3}$ $\quad w = R^{-1}P = \begin{bmatrix}-1\\-2\end{bmatrix}$

b. $w^T x_0 = \begin{bmatrix}-1&-2\end{bmatrix}\begin{bmatrix}0\\-1\end{bmatrix} = 2$

$w^T x_1 = \begin{bmatrix}-1&-2\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix} = -1$

$w^T x_2 = \begin{bmatrix}-1&-2\end{bmatrix}\begin{bmatrix}-1\\1\end{bmatrix} = -1$

matches $d(0)$, $d(1)$ and $d(2)$ ✓

c. $L=2$

$R = \begin{bmatrix}1&0\\0&1\end{bmatrix}$, $P = \begin{bmatrix}-1\\-2\end{bmatrix}$, $R^{-1} = \begin{bmatrix}1&0\\0&1\end{bmatrix}$, $w = R^{-1}P = \begin{bmatrix}1&0\\0&1\end{bmatrix}\begin{bmatrix}-1\\-2\end{bmatrix} = \begin{bmatrix}-1\\-2\end{bmatrix}$

matches prev so it will match $d(0)$ and $d(1)$

$L=4$

d. $x_3 = x_0$, $d(3) = d(0)$ b/c of periodic-ness

$R = \begin{bmatrix}2&-1\\-1&3\end{bmatrix}$, $P = \begin{bmatrix}0\\-5\end{bmatrix}$, $R^{-1} = \begin{bmatrix}3&1\\1&2\end{bmatrix} \cdot \frac{1}{5}$

$w = R^{-1}P = \frac{1}{5}\begin{bmatrix}3&1\\1&2\end{bmatrix}\begin{bmatrix}0\\-5\end{bmatrix} = \begin{bmatrix}-1\\-2\end{bmatrix}$ ← same again

5. a.

$x(0)=1$ $\quad d(0)=2$

$x(1)=0$ $\quad d(1)=-1$

$x(2)=-1$ $\quad d(2)=-1$

$x(3)=0$

$x_0 = \begin{bmatrix}1\\0\\-1\end{bmatrix}$, $x_1 = \begin{bmatrix}0\\1\\0\end{bmatrix}$, $x_2 = \begin{bmatrix}-1\\0\\1\end{bmatrix}$, $x_3 = \begin{bmatrix}0\\-1\\0\end{bmatrix}$

x_4, x_8 $\quad x_5, x_9$ $\quad x_6, x_{10}$ $\quad x_7, x_{11}$

$R_0 = R_4 = R_8 = \begin{bmatrix}1\\0\\-1\end{bmatrix}\begin{bmatrix}1 & 0 & -1\end{bmatrix} = \begin{bmatrix}1 & 0 & -1\\0 & 0 & 0\\-1 & 0 & 1\end{bmatrix}$

$R_1 = R_5 = R_9 = \begin{bmatrix}0\\1\\0\end{bmatrix}\begin{bmatrix}0 & 1 & 0\end{bmatrix} = \begin{bmatrix}0 & 0 & 0\\0 & 1 & 0\\0 & 0 & 0\end{bmatrix}$

$R_2 = R_6 = R_{10} = \begin{bmatrix}-1\\0\\1\end{bmatrix}\begin{bmatrix}-1 & 0 & 1\end{bmatrix} = \begin{bmatrix}1 & 0 & -1\\0 & 0 & 0\\-1 & 0 & 1\end{bmatrix}$

$R_3 = R_7 = R_{11} = \begin{bmatrix}0\\-1\\0\end{bmatrix}\begin{bmatrix}0 & -1 & 0\end{bmatrix} = \begin{bmatrix}0 & 0 & 0\\0 & 1 & 0\\0 & 0 & 0\end{bmatrix}$

b/c of periodicity

$R2 \begin{bmatrix}2 & 0 & -2\\0 & 2 & 0\\-2 & 0 & 2\end{bmatrix} \cdot 3$ ← inverse still doesn't exist.

3 of problem is repeated

b. Now d and x are diff periods, won't be consistent across mappings

c. $M=2$

$R_0 = R_4 = R_8 = \begin{pmatrix}1 & 0\\0 & 0\end{pmatrix}$

$R_1 = R_5 = R_9 = \begin{pmatrix}0 & 0\\0 & 1\end{pmatrix}$ $\quad R = \begin{bmatrix}2 & 0\\0 & 2\end{bmatrix} \cdot 3 \quad R^{-1} = \begin{pmatrix}\frac{1}{6} & 0\\0 & \frac{1}{6}\end{pmatrix}$

$R_2 = R_6 = R_{10} = \begin{pmatrix}1 & 0\\0 & 0\end{pmatrix}$

$R_3 = R_7 = R_{11} = \begin{pmatrix}0 & 0\\0 & 1\end{pmatrix}$

$$\frac{1}{12}\left(2\begin{bmatrix}1\\0\\-1\end{bmatrix} - \begin{bmatrix}0\\1\\0\end{bmatrix} - \begin{bmatrix}-1\\0\\1\end{bmatrix} + 2\begin{bmatrix}0\\-1\\0\end{bmatrix} - \begin{bmatrix}1\\0\\-1\end{bmatrix} - \begin{bmatrix}0\\1\\0\end{bmatrix} + 2\begin{bmatrix}-1\\0\\1\end{bmatrix} - \begin{bmatrix}0\\-1\\0\end{bmatrix} - \begin{bmatrix}1\\0\\-1\end{bmatrix}\right.$$

$$\left. + 2\begin{bmatrix}0\\1\\0\end{bmatrix} - \begin{bmatrix}-1\\0\\1\end{bmatrix} - \begin{bmatrix}0\\-1\\0\end{bmatrix}\right) = P$$

$= \frac{1}{12}\begin{bmatrix}0\\0\end{bmatrix}$ ← only care about top 2

$P = \begin{bmatrix}0\\0\end{bmatrix}$

$W = R^{-1}P = \begin{bmatrix}0\\0\end{bmatrix}$

best choice for W

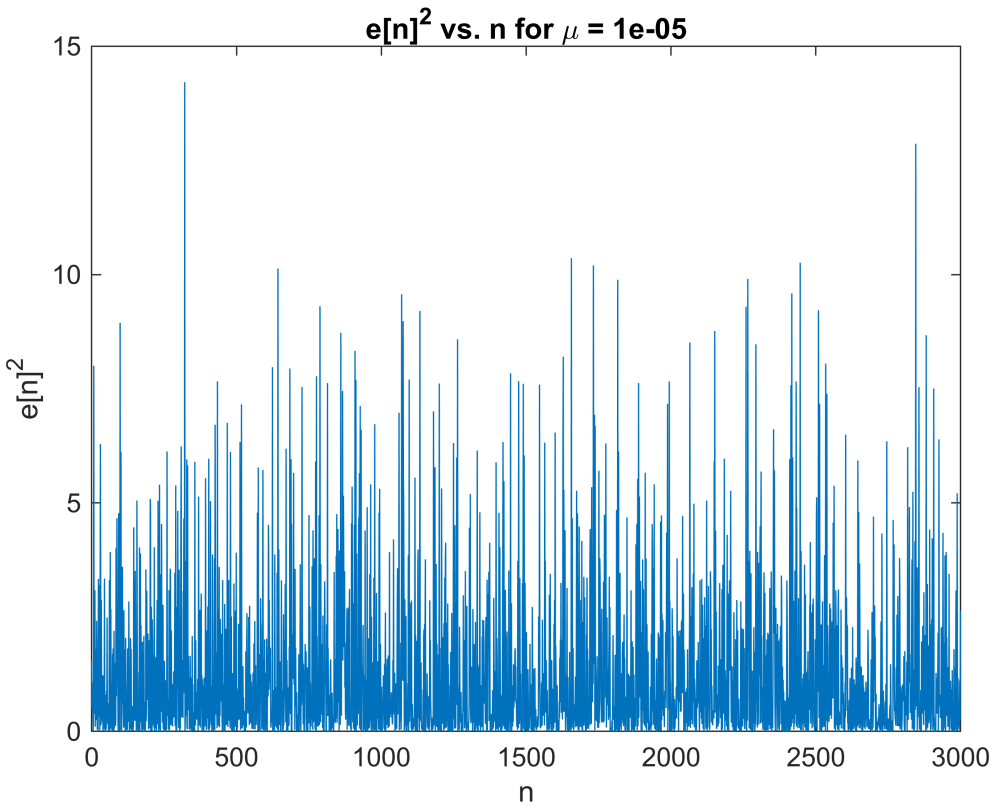
$\boxed{W = \begin{bmatrix}0\\0\end{bmatrix}}$

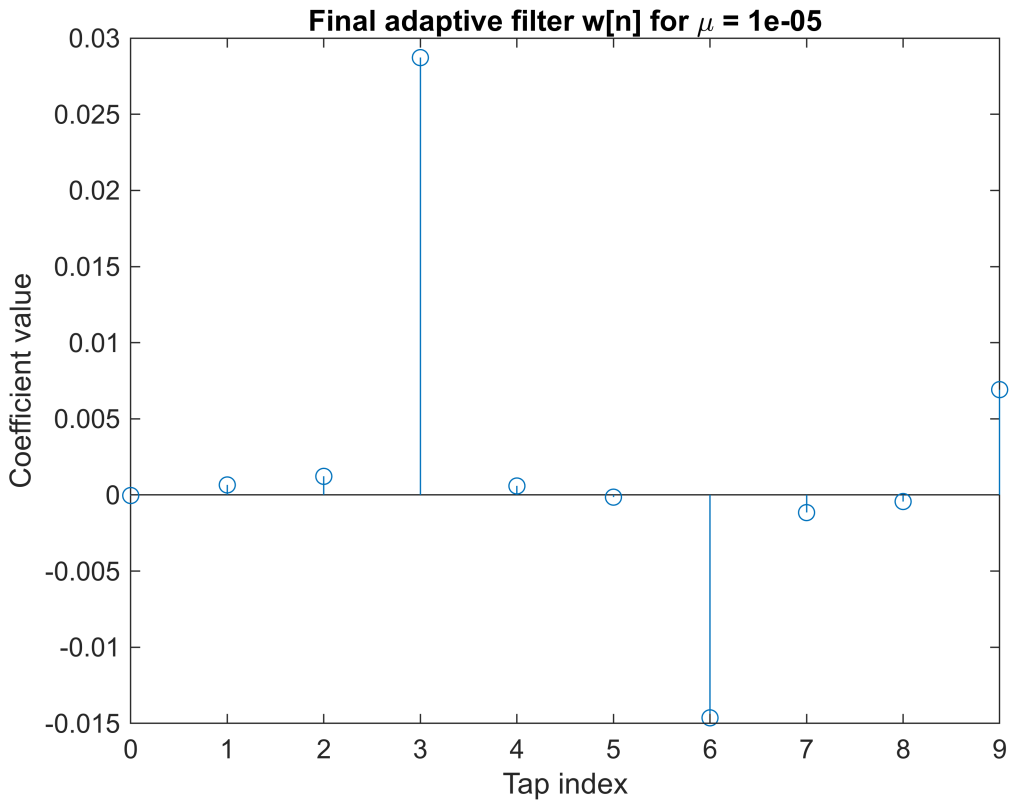
6) (15 points) Implement an adaptive filter in the System Identification architecture using MATLAB to match the output of an unknown system h = [1, 0, 0, -0.5, 0, 0, 0.25]. Use a filter length of 10 for the adaptive filter w and use the MATLAB function randn(1,N) to create the input sequence for both the unknown system and the adaptive system. N should be set equal to the number of output samples needed plus (M-1) where M is the filter length. The desired signal will be the convolution of the unknown filter h and this input sequence.

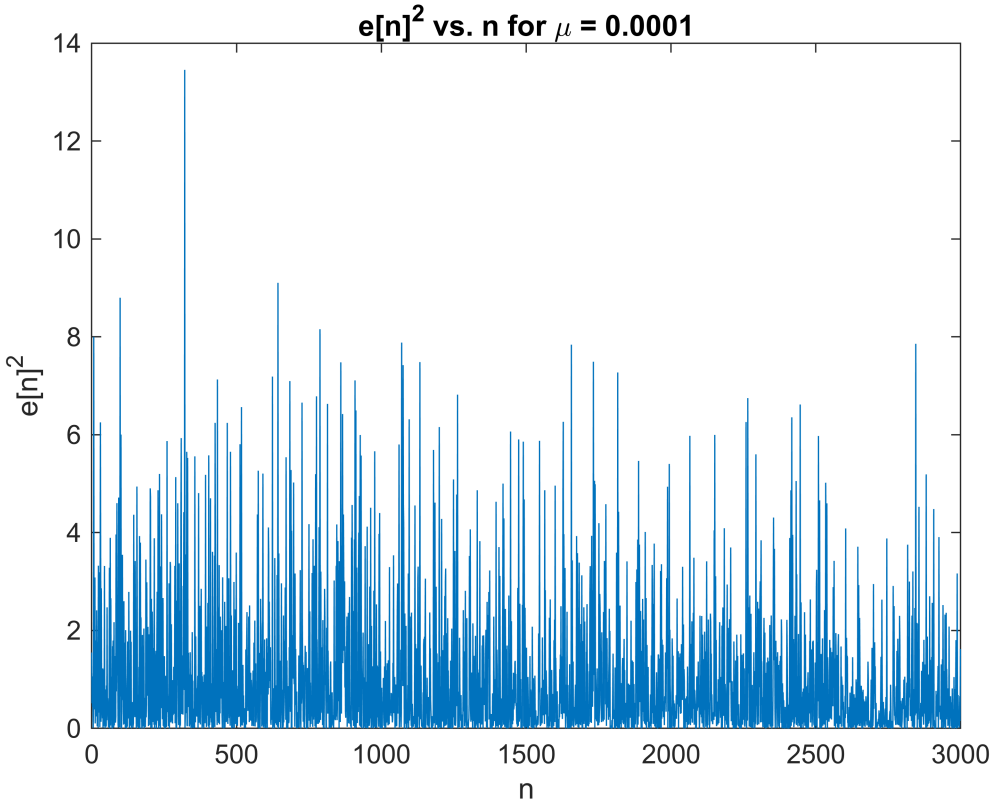
a) For three separate adaptation constants, $\mu = 10^{-5}, \mu = 10^{-4}, \mu = 10^{-3}$, adapt the filter for 3000 samples of the desired signal. For each adaptation constant, plot values of $e[n]^2$ for $0 \leq n \leq 2999$, and also make a stem plot of the adaptive filter values after 3000 updates.
b) Compare the final value of w[0] of the three adapted filters to h[0] and to each other. How is the final value of w[0] related to the adaptation constant? What would you expect the value of w[0] to be for $\mu = 10^{-6}$?
c) For each of the three values of the adaptation constant, compare the general distribution of values of $e[n]^2$ for n near 3000 to the initial range of values of $e[n]^2$ for n near 0. How is this related to your answer to part (b)?

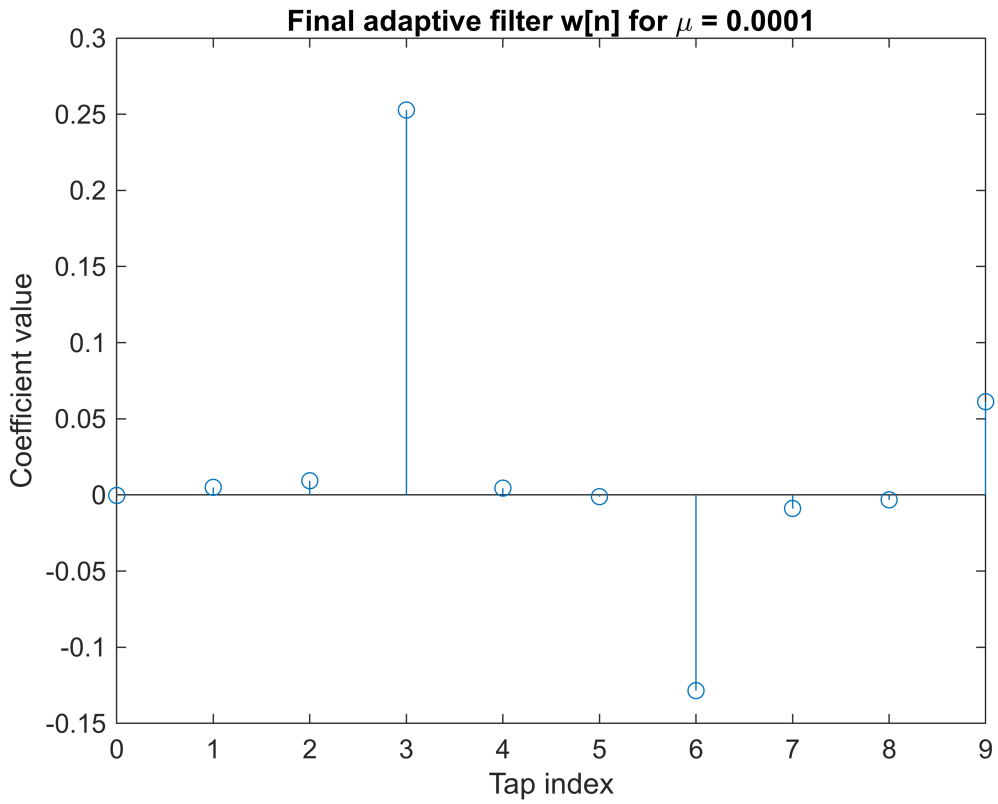
```
h = [1, 0, 0, -0.5, 0, 0, .25];
M = 10;
mu_vals = [1e-5, 1e-4, 1e-3];
N = 3000 + M - 1;

x = randn(1, N);

d = conv(x, h, 'valid');  % length = 3000

for k = 1:length(mu_vals)
    mu = mu_vals(k);

    w = zeros(M, 1); % adaptive filter
    e = zeros(1, 3000); % error
    y = zeros(1, 3000); % output


    for n = 1:3000
        x_n = x(n+M-1:-1:n).';
        y(n) = w' * x_n;
        e(n) = d(n) - y(n);
        w = w + mu * x_n * e(n);
    end

    % plot error squared
    figure;
    plot(0:2999, e.^2);
```

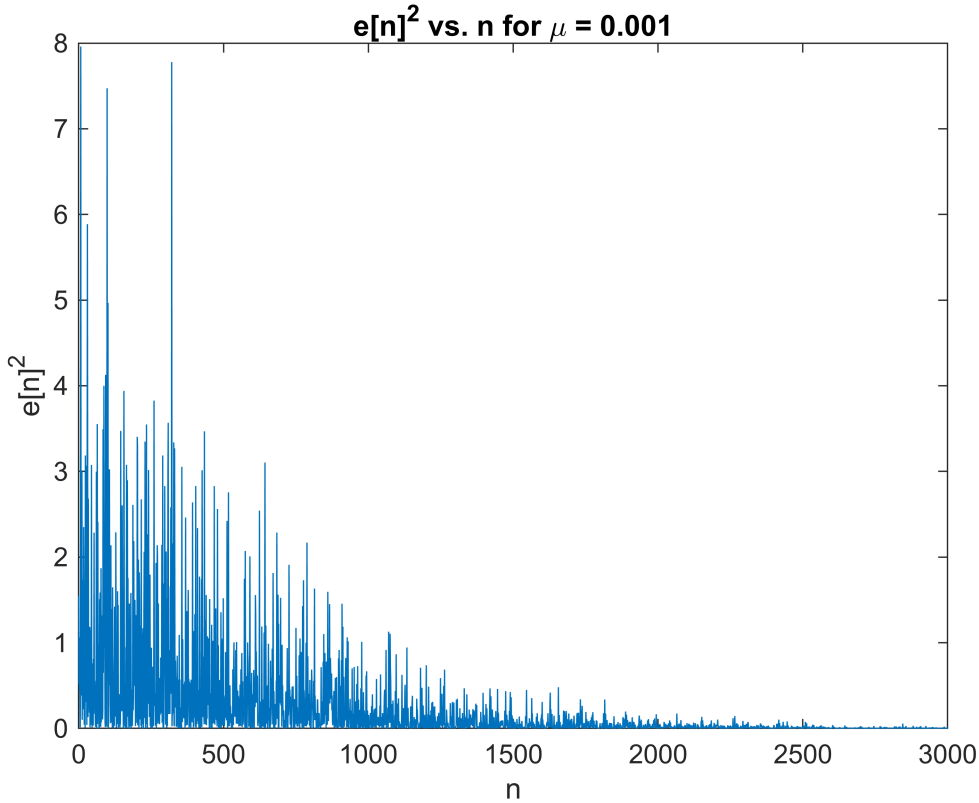
```
    title(['e[n]^2 vs. n for \mu = ', num2str(mu)]);
    xlabel('n');
    ylabel('e[n]^2');

    % stem plot of final filter coefficients
    figure;
    stem(0:M-1, w);
    title(['Final adaptive filter w[n] for \mu = ', num2str(mu)]);
    xlabel('Tap index');
    ylabel('Coefficient value');
end
```

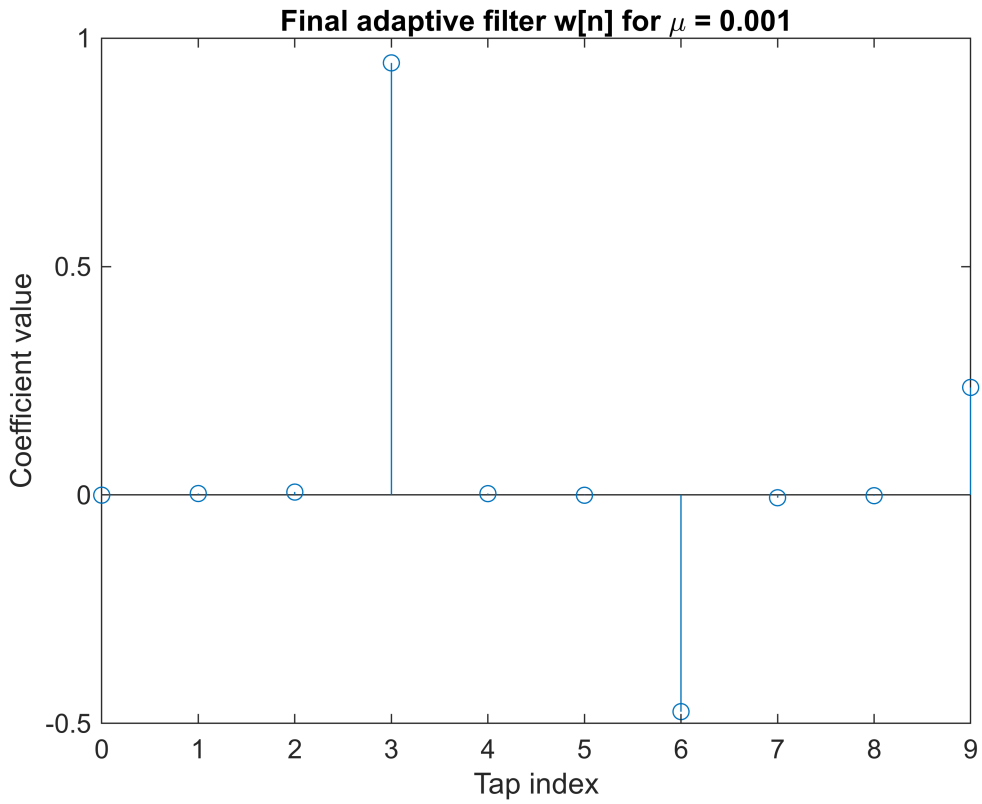



Final adaptive filter w[n] for μ = 1e-05
Coefficient value
Tap index
0.03
0.025
0.02
0.015
0.01
0.005
0
-0.005
-0.01
-0.015
0
1
2
3
4
5
6
7
8
9


e[n]^2 vs. n for μ = 0.0001
e[n]^2
n
14
12
10
8
6
4
2
0
0
500
1000
1500
2000
2500
3000


Final adaptive filter w[n] for μ = 0.0001
Coefficient value
Tap index
0.3
0.25
0.2
0.15
0.1
0.05
0
-0.05
-0.1
-0.15
0
1
2
3
4
5
6
7
8
9


e[n]^2 vs. n for μ = 0.001
e[n]^2
n
8
7
6
5
4
3
2
1
0
0
500
1000
1500
2000
2500
3000



mu=.001 gives us the closest w[0] value as it's almost 1, while the others are ~.25 and ~.03. A higher mu leads to better adapting. I would expect w[0] to be even lower for mu = 1e-6.

We also see as mu increases, the error function drops off way more. It's really only visible for mu=.001.

7) (10 points) The stable range of the value of the adaptation constant is related to the power level of the input.

a) If the input is a*randn(1,N) instead of randn(1,N), how will this affect the outer product $x_n \cdot (x_n)^T$?

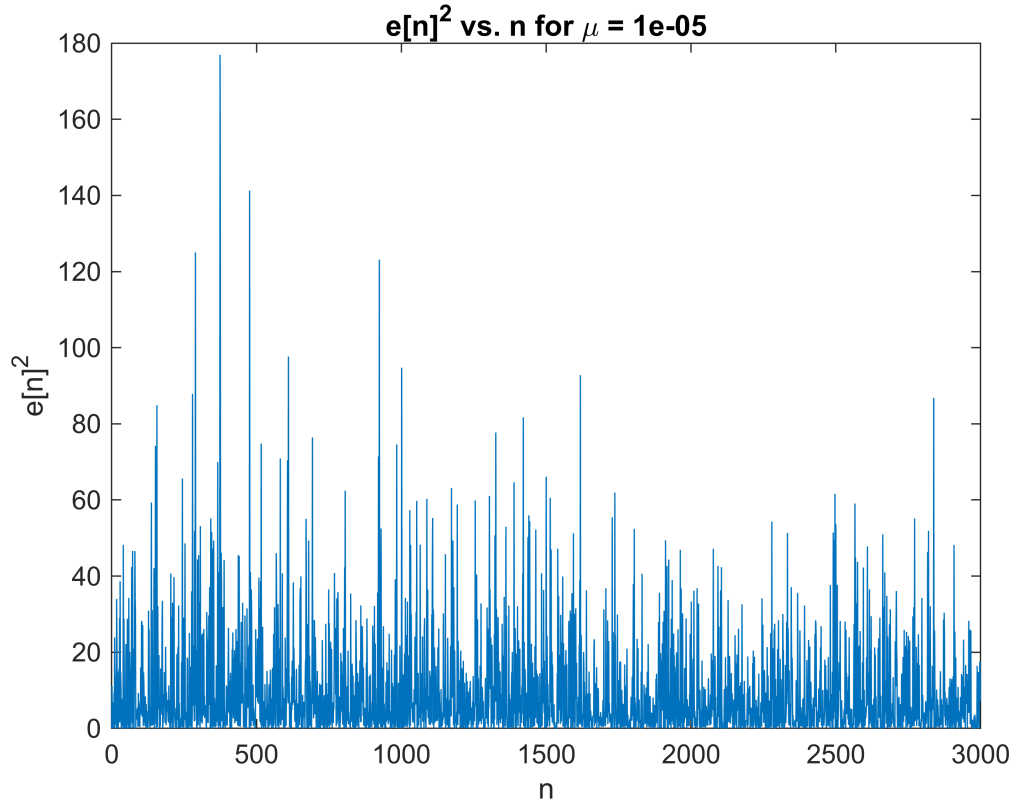
The outer product is now axn * (axn)^T = a^2 * (xn * (xn)^T)

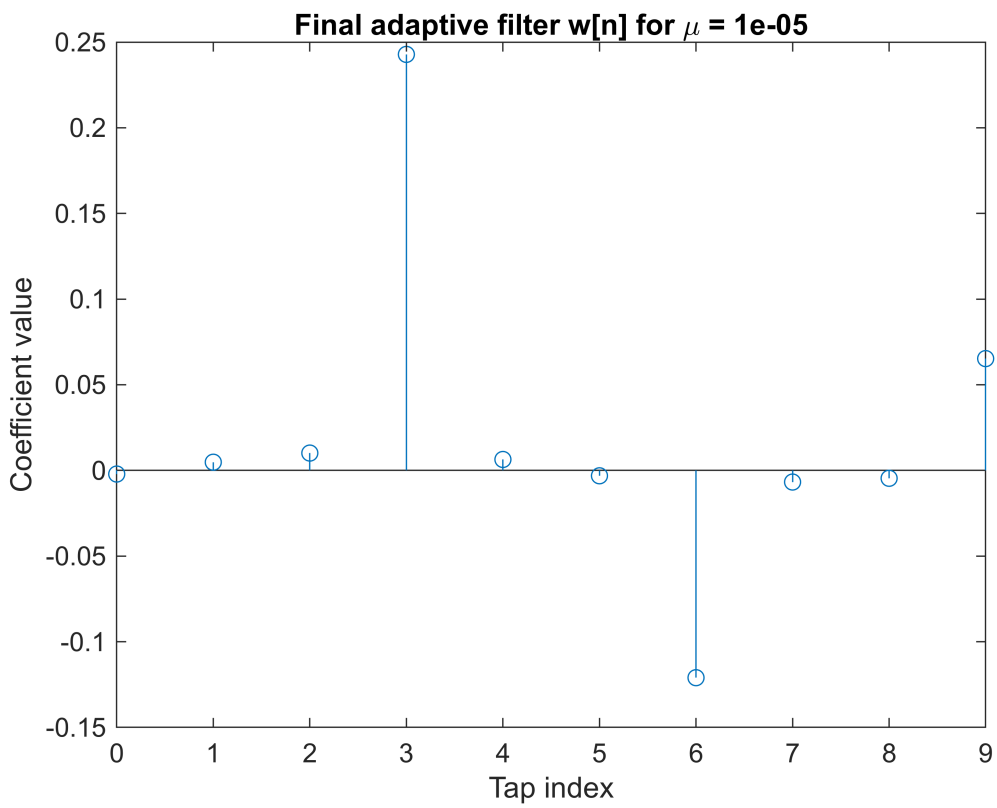
It gets scaled by a^2.

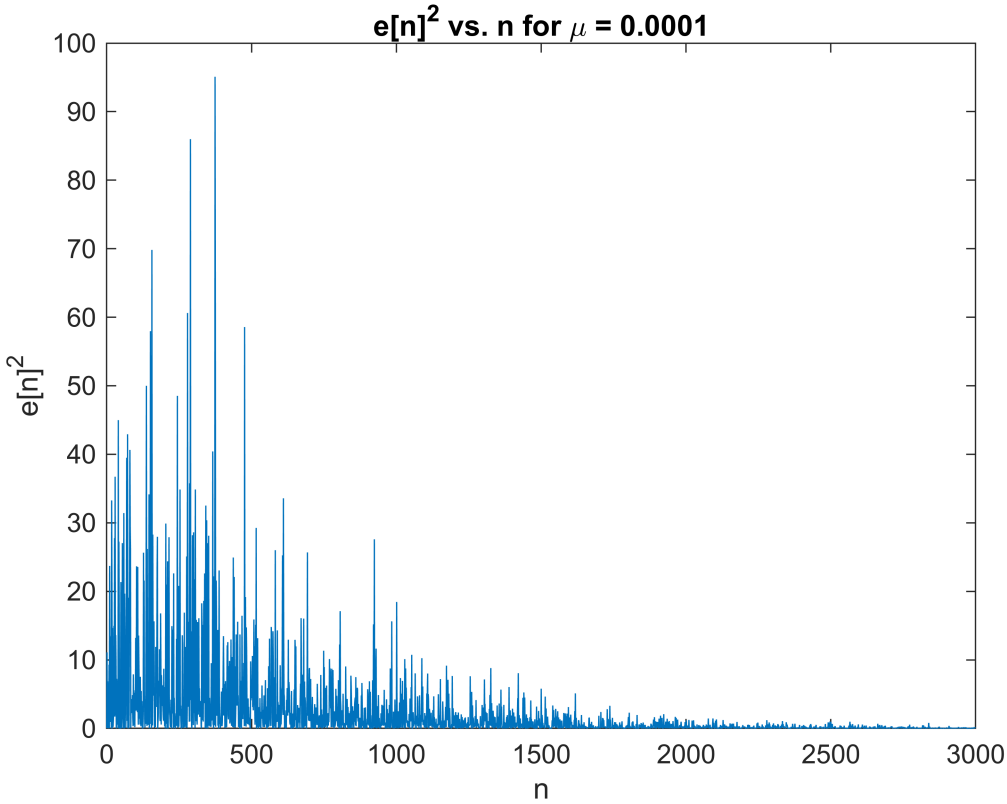
b) Repeat Problem 6a using an input of a*randn(1,N) with $a = \sqrt{10}$. What is the relationship between the adaptation constants which produce similar results here and in Problem 6a ?

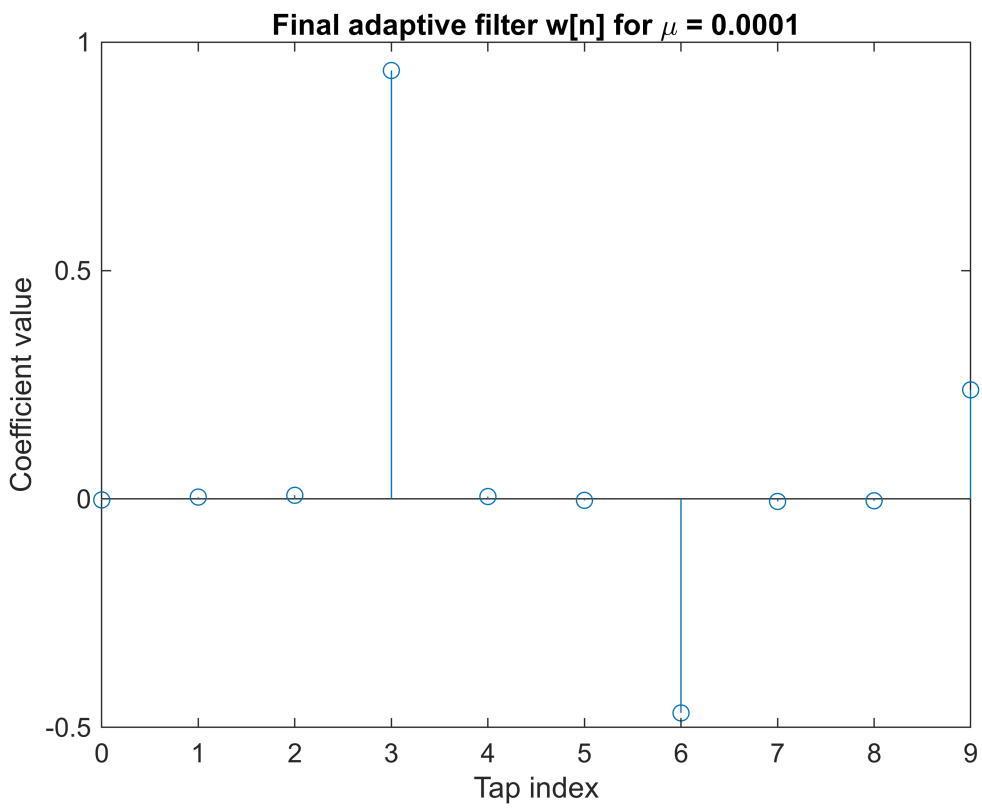
```
h = [1, 0, 0, -0.5, 0, 0, .25];
M = 10;
mu_vals = [1e-5, 1e-4, 1e-3];
N = 3000 + M - 1;
a = sqrt(10);
```

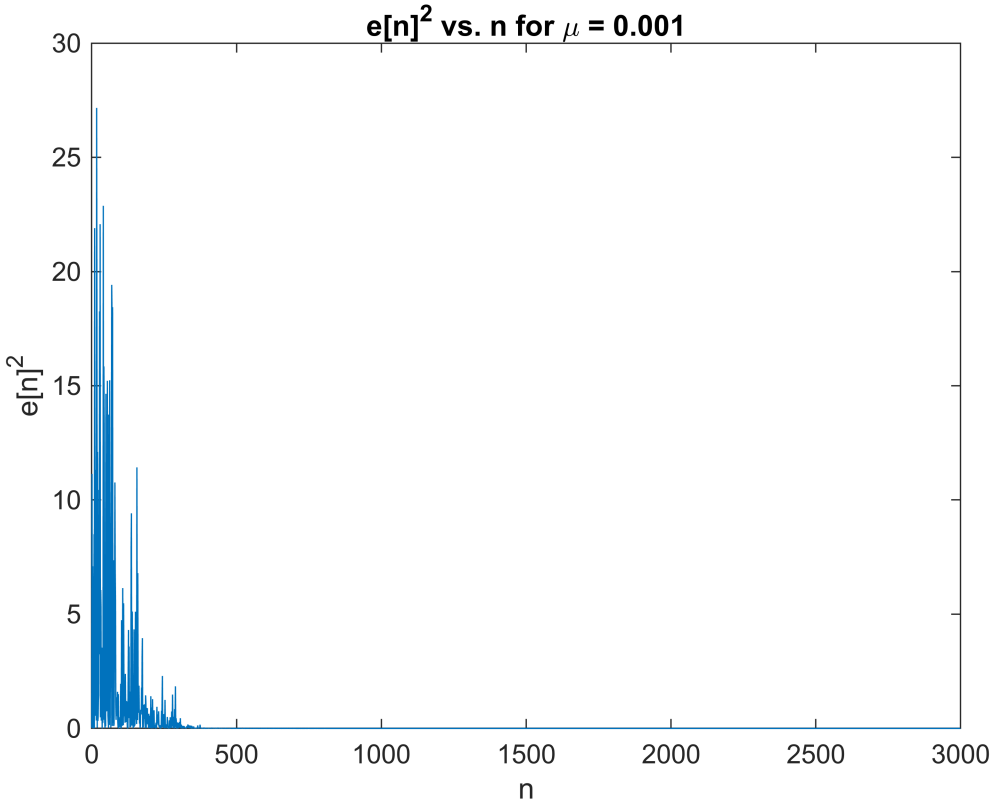
```
x = a*randn(1, N);

d = conv(x, h, 'valid');  % length = 3000

for k = 1:length(mu_vals)
    mu = mu_vals(k);

    w = zeros(M, 1); % adaptive filter
    e = zeros(1, 3000); % error
    y = zeros(1, 3000); % output


    for n = 1:3000
        x_n = x(n+M-1:-1:n).';
        y(n) = w' * x_n;
        e(n) = d(n) - y(n);
        w = w + mu * x_n * e(n);
    end

    % plot error squared
    figure;
    plot(0:2999, e.^2);
    title(['e[n]^2 vs. n for \mu = ', num2str(mu)]);
    xlabel('n');
    ylabel('e[n]^2');

    % stem plot of final filter coefficients
    figure;
    stem(0:M-1, w);
    title(['Final adaptive filter w[n] for \mu = ', num2str(mu)]);
    xlabel('Tap index');
    ylabel('Coefficient value');
end
```

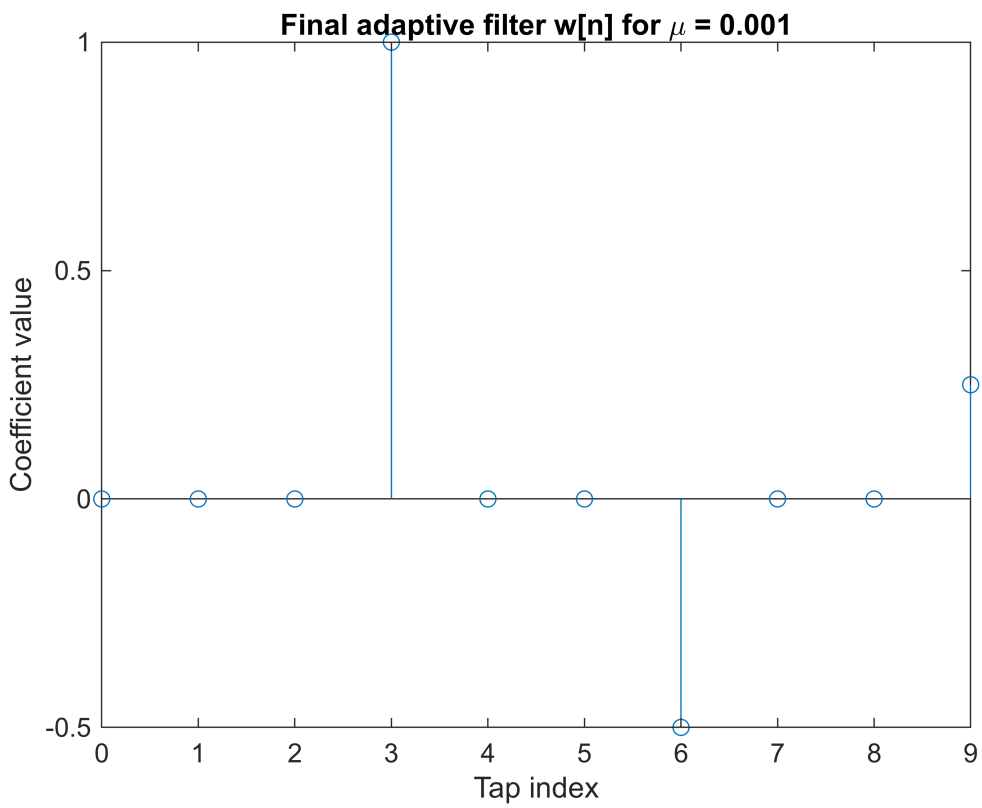

e[n]^2 vs. n for μ = 1e-05
e[n]^2
n
0
20
40
60
80
100
120
140
160
180
0
500
1000
1500
2000
2500
3000


Final adaptive filter w[n] for μ = 1e-05
Coefficient value
Tap index
0.25
0.2
0.15
0.1
0.05
0
-0.05
-0.1
-0.15
0
1
2
3
4
5
6
7
8
9


e[n]^2 vs. n for μ = 0.0001
e[n]^2
n
0
10
20
30
40
50
60
70
80
90
100
0
500
1000
1500
2000
2500
3000


Final adaptive filter w[n] for μ = 0.0001
Coefficient value
Tap index
1
0.5
0
-0.5
0
1
2
3
4
5
6
7
8
9

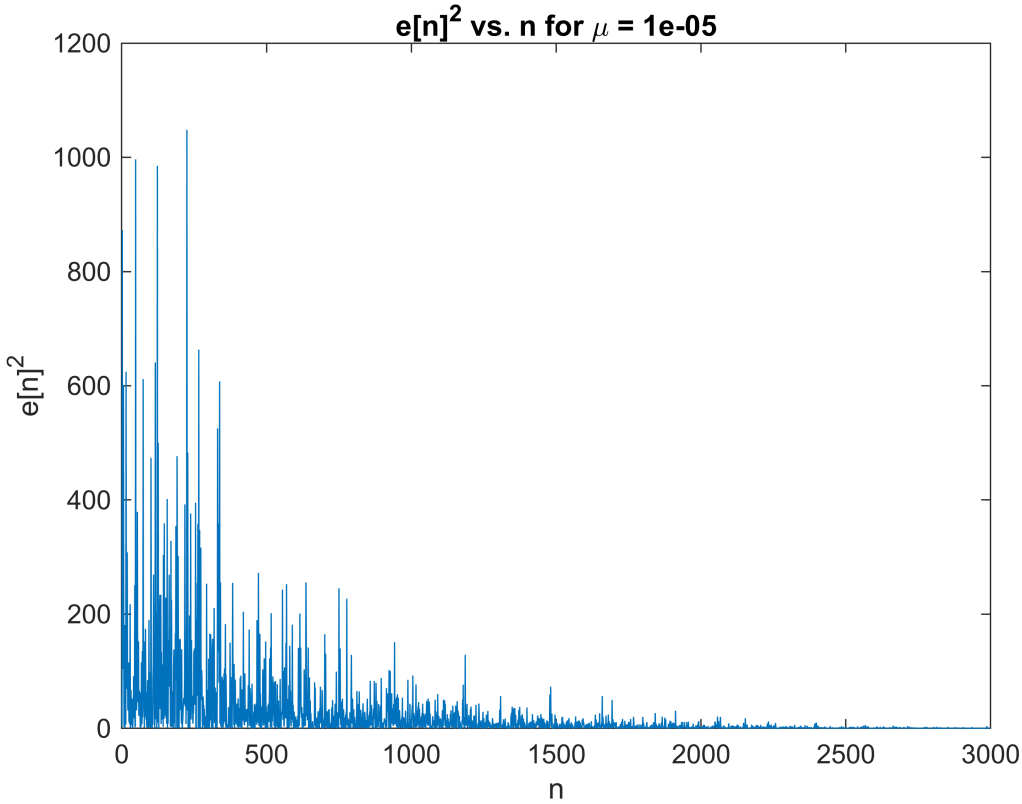


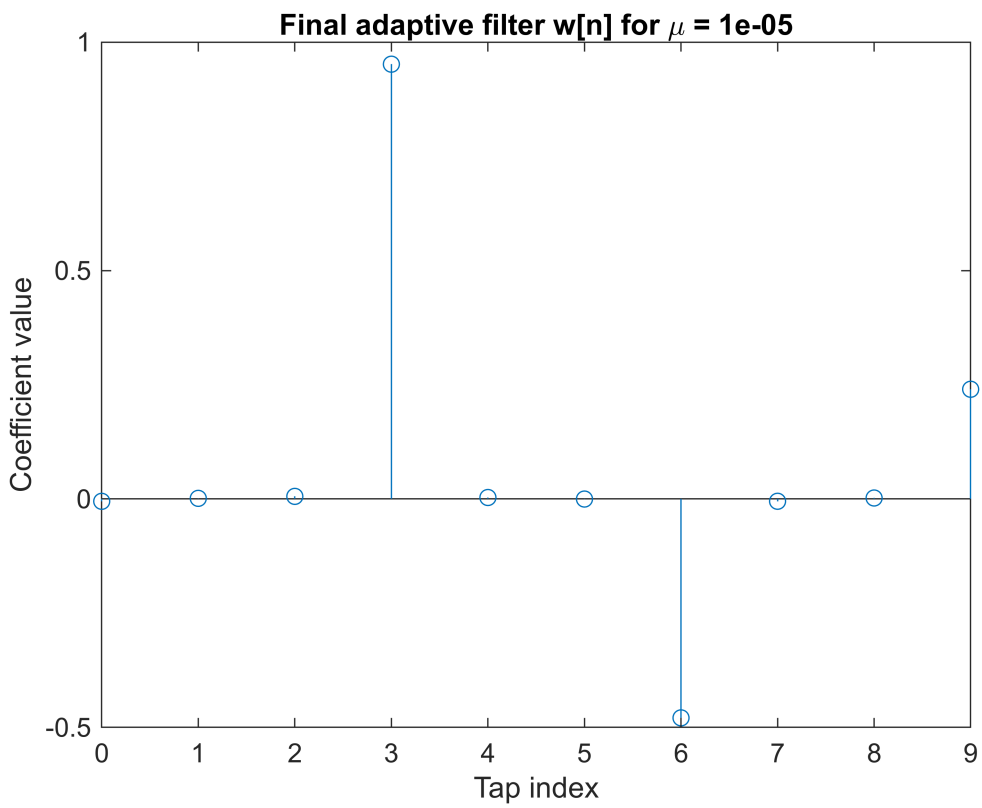


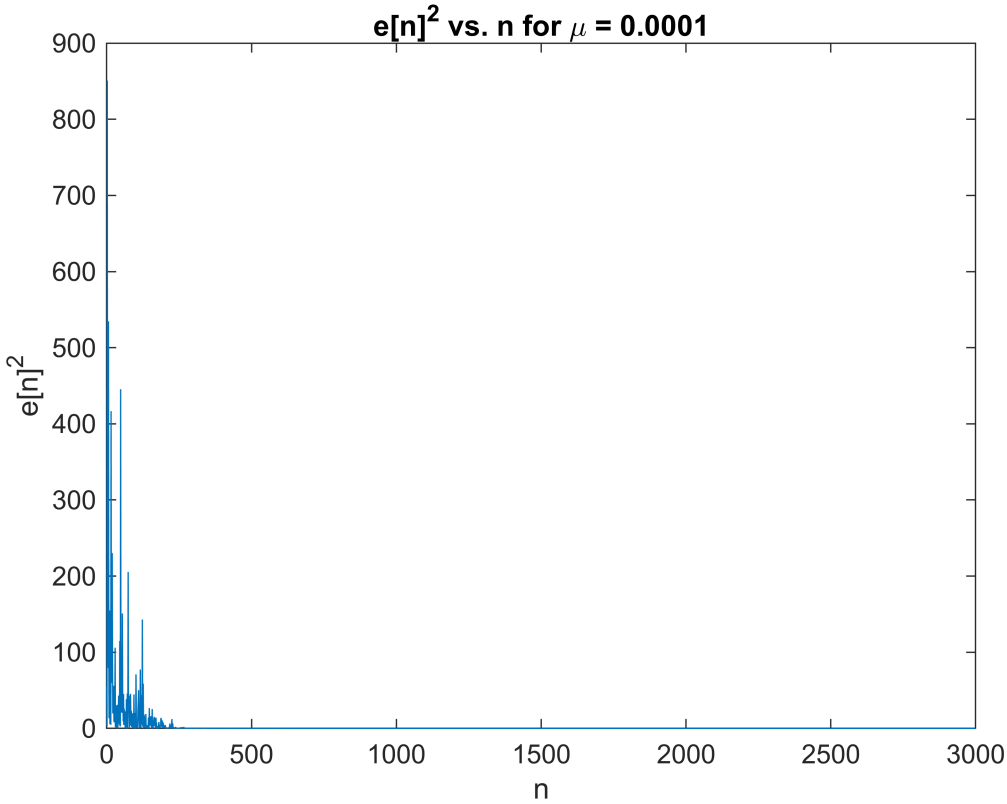
a helped the smaller mu values achieve more accurate weights and decrease the error function more rapidly. I presume the bigger the adaptation constant, the more accurate our weights can be.

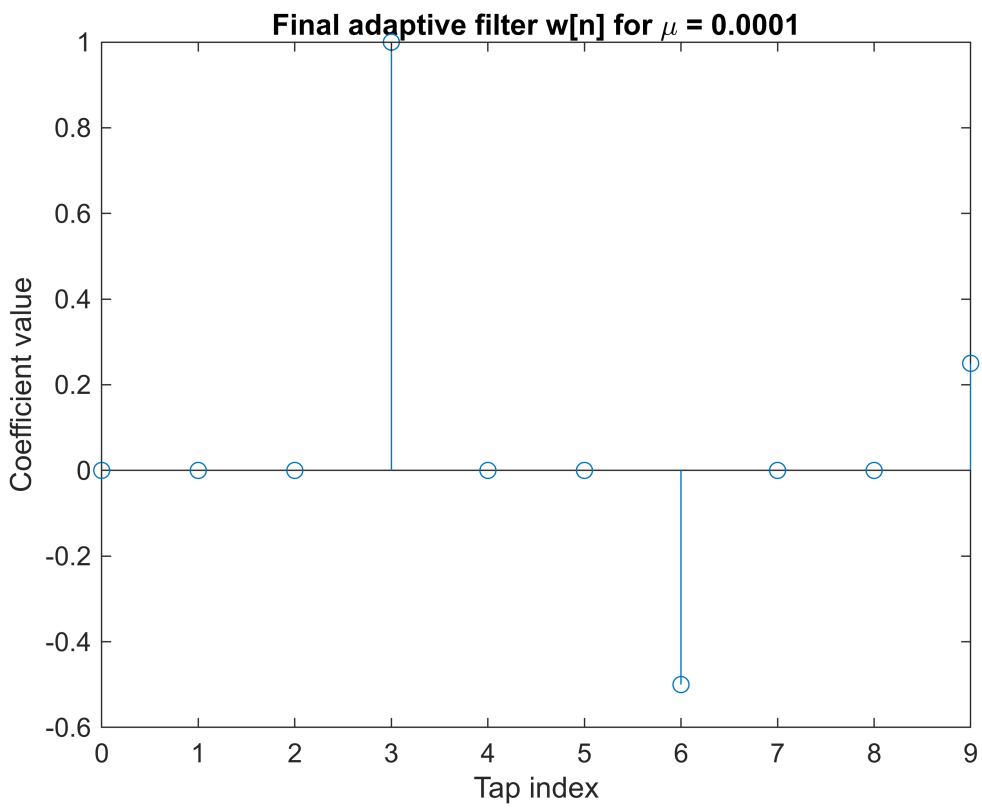
c) Repeat part (b) for $a = 10$.

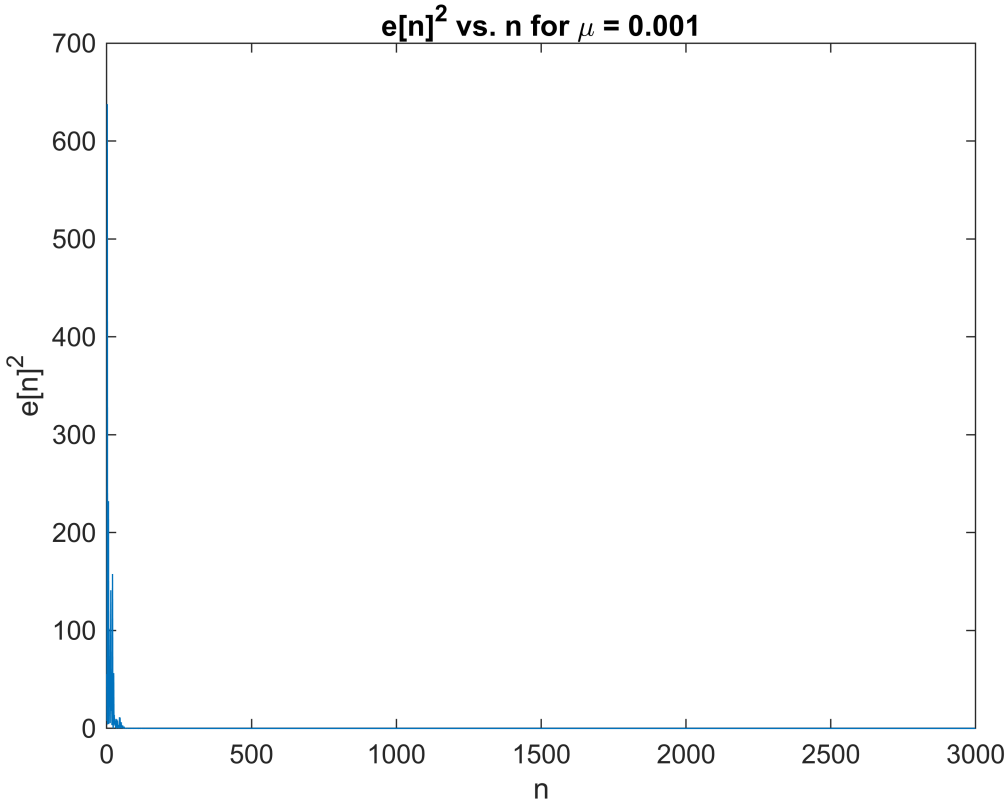
```
h = [1, 0, 0, -0.5, 0, 0, .25];
M = 10;
mu_vals = [1e-5, 1e-4, 1e-3];
N = 3000 + M - 1;
a = 10;

x = a*randn(1, N);

d = conv(x, h, 'valid');  % length = 3000

for k = 1:length(mu_vals)
    mu = mu_vals(k);

    w = zeros(M, 1); % adaptive filter
    e = zeros(1, 3000); % error
    y = zeros(1, 3000); % output


    for n = 1:3000
        x_n = x(n+M-1:-1:n).';
        y(n) = w' * x_n;
        e(n) = d(n) - y(n);
        w = w + mu * x_n * e(n);
    end

    % plot error squared
    figure;
    plot(0:2999, e.^2);
    title(['e[n]^2 vs. n for \mu = ', num2str(mu)]);
    xlabel('n');
    ylabel('e[n]^2');

    % stem plot of final filter coefficients
    figure;
    stem(0:M-1, w);
    title(['Final adaptive filter w[n] for \mu = ', num2str(mu)]);
    xlabel('Tap index');
    ylabel('Coefficient value');
end
```

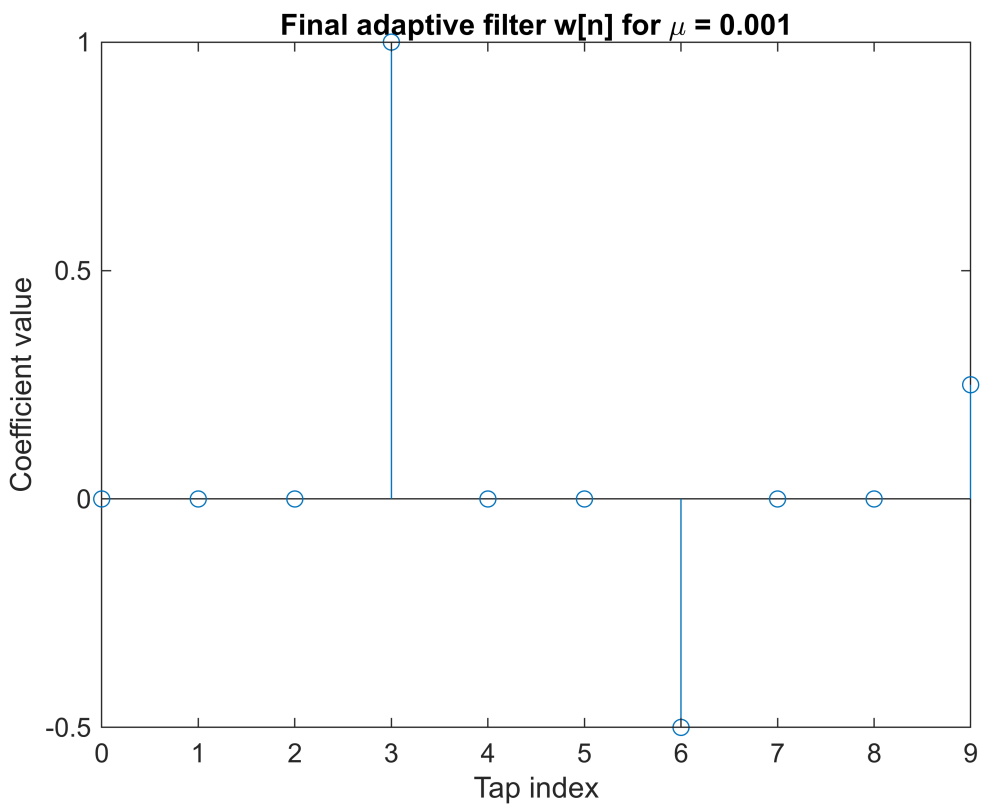

e[n]^2 vs. n for μ = 1e-05
1200
1000
800
600
400
200
0
e[n]^2
0
500
1000
1500
2000
2500
3000
n


Final adaptive filter w[n] for μ = 1e-05
1
0.5
0
-0.5
Coefficient value
0
1
2
3
4
5
6
7
8
9
Tap index


e[n]^2 vs. n for μ = 0.0001
e[n]^2
n
900
800
700
600
500
400
300
200
100
0
0
500
1000
1500
2000
2500
3000


Final adaptive filter w[n] for μ = 0.0001
Coefficient value
Tap index
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
0
1
2
3
4
5
6
7
8
9





As predicted, the higher adaptation constant led to more accurate weights and a much quicker dropoff on the error function output for all mus.